AWS CLI Installation

Command

```
aws --version
```

Description

This command checks whether **AWS CLI is installed on the system** or not.
If AWS CLI is installed, it will display the **version of AWS CLI**.

Example output:

```
aws-cli/2.15.0 Python/3.x Windows/10
```

Configure AWS CLI

Command

```
aws configure
```

Description

This command is used to **connect AWS CLI with your AWS account**.

When you run this command, it will ask for four values.

Example:

```
AWS Access Key ID:
AWS Secret Access Key:
Default region name:
Default output format:
```

Explanation of fields:

Field	Description
Access Key ID	Used for authentication
Secret Access Key	Used for security
Default Region	AWS region where resources will be created
Output Format	Format of output like json, table

Example configuration:

```
AWS Access Key ID: AKIA123456
AWS Secret Access Key: xyz123abc
Default region name: ap-south-1
Default output format: json
```

Here **ap-south-1 represents the Mumbai region**.

3. Show Help in AWS CLI

Command

```
aws help
```

Description

This command displays **all AWS services and commands supported by AWS CLI**.

Example services displayed:

- ec2
- s3
- iam
- lambda
- cloudwatch
- rds
- dynamodb

4. Show Help for a Specific Service

Command

```
aws ec2 help
```

Description

This command shows **all operations related to the EC2 service**.

Example operations:

- run-instances
- stop-instances
- start-instances
- describe-instances

5. List All EC2 Instances

Command

```
aws ec2 describe-instances
```

Description

This command shows **all EC2 instances available in your AWS account**.

The output includes:

- Instance ID
- Instance state
- Instance type
- Public IP address
- Region information

6. Create an EC2 Instance

Command

```
aws ec2 run-instances --image-id ami-xxxxxxxx --count 1 --instance-type
t2.micro --region ap-south-1
```

Description

This command creates a **new EC2 virtual server** in AWS.

Explanation of parameters:

Parameter	Meaning
run-instances	command to launch instance
image-id	AMI (Operating System Image)
count	number of instances
instance-type	server configuration
region	AWS region

Example region:

```
ap-south-1
```

This launches the instance in **Mumbai region**.

7. Start an EC2 Instance

Command

```
aws ec2 start-instances --instance-ids i-123456789
```

Description

This command **starts a stopped EC2 instance**.

8. Stop an EC2 Instance

Command

```
aws ec2 stop-instances --instance-ids i-123456789
```

Description

This command **stops a running EC2 instance**.

9. Terminate an EC2 Instance

Command

```
aws ec2 terminate-instances --instance-ids i-123456789
```

Description

This command **permanently deletes an EC2 instance**.

What is a Profile in AWS CLI?

A **profile** in AWS CLI is a **named set of credentials and configuration** (Access Key, Secret Key, Region, Output format).

Profiles allow you to **manage multiple AWS accounts or IAM users** from the same AWS CLI installation.

Example Use Case

You may have:

- **Personal AWS account**
- **Company AWS account**
- **Testing account**

Instead of changing credentials every time, you create **different profiles**.

Example profiles:

- default
- admin
- developer
- testing

Create the Default Profile

Command

```
aws configure
```

Description

This creates the **default profile** in AWS CLI.

It will ask:

```
AWS Access Key ID:
AWS Secret Access Key:
Default region name:
Default output format:
```

Example:

```
AWS Access Key ID: AKIAxxxx
AWS Secret Access Key: xxxxxx
Default region name: ap-south-1
Default output format: json
```

3. Create Multiple Profiles

To create another profile, use:

Command

```
aws configure --profile profilename
```

Example

```
aws configure --profile admin
```

It will ask the same details:

```
AWS Access Key ID:
AWS Secret Access Key:
Default region name:
Default output format:
```

Now a **new profile named "admin"** is created.

---

4. Example of Multiple Profiles

You can create profiles like:

```
aws configure --profile developer
aws configure --profile testing
aws configure --profile production
```

Now your CLI has multiple accounts configured.

---

5. Use a Specific Profile

To run a command using a specific profile, use:

Command

```
aws <service> <command> --profile profilename
```

Example

```
aws ec2 describe-instances --profile admin
```

This command will use **admin profile credentials**.

---

6. List All Configured Profiles

Command

```
aws configure list-profiles
```

Example Output

```
default
admin
developer
testing
```

This shows all profiles configured in AWS CLI.

---

7. View Profile Configuration

Command

```
aws configure list --profile profilename
```

Example

```
aws configure list --profile admin
```

This shows:

- Access key
- Region
- Output format

---

8. Edit a Profile

To update profile configuration:

```
aws configure --profile profilename
```

Example:

```
aws configure --profile developer
```

You can change keys, region, or output format.

---

9. Delete a Profile

Profiles are stored in configuration files.

Location:

```
C:\Users\YourUser\.aws\
```

Files:

- **credentials**
- **config**

To delete a profile, open these files and remove the profile section.

Example:

```
[admin]
aws_access_key_id=XXXX
aws_secret_access_key=XXXX
```

Delete this section to remove the profile.

---

10. Example Practical Workflow

Step 1: Configure default account

```
aws configure
```

Step 2: Configure another account

```
aws configure --profile admin
```

Step 3: Use admin profile

```
aws ec2 describe-instances --profile admin
```

Step 4: List profiles

```
aws configure list-profiles
```

---

11. Benefits of Using Multiple Profiles

- Manage **multiple AWS accounts easily**
- Avoid reconfiguring credentials repeatedly
- Useful for **developers, DevOps, and administrators**
- Helps separate **production and testing environments**

Summary

Command	Purpose
`aws configure`	Create default profile
`aws configure --profile name`	Create new profile
`aws configure list-profiles`	Show all profiles
`aws configure list --profile name`	Show profile details
`--profile name`	Use specific profile

What are AWS Credentials?

AWS Credentials are security keys used to **authenticate and access AWS services**.

When using **AWS CLI or API**, AWS needs credentials to verify **who you are**.

AWS credentials mainly include:

1. **Access Key ID**
2. **Secret Access Key**

Example:

```
Access Key ID: AKIAIOSFODNN7EXAMPLE
Secret Access Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

These credentials allow users or applications to **access AWS services such as EC2, S3, IAM, etc.**

Where AWS CLI Stores Credentials

When you run:

```
aws configure
```

AWS CLI saves credentials in a hidden folder.

Location (Windows)

```
C:\Users\YourUsername\.aws\
```

Inside this folder there are two files:

```
credentials
config
```

Example of **credentials file**

```
[default]
aws_access_key_id = AKIA12345
aws_secret_access_key = abcxyz123
```

Example of **config file**

```
[default]
region = ap-south-1
output = json
```

3. Why Credentials Are Risky

If someone gets your **Access Key and Secret Key**, they can:

- Create EC2 instances
- Delete resources
- Access S3 buckets
- Use your AWS account

So AWS recommends using **IAM Roles instead of storing credentials**.

4. What is an IAM Role?

An **IAM Role** is a secure way to give permissions to AWS services **without using permanent credentials**.

Unlike IAM users:

- A **role does not have a password**
- A **role does not have access keys**
- It provides **temporary credentials**

Roles are mostly used by:

- EC2 instances
- Lambda functions
- AWS services
- Cross-account access

5. Example of IAM Role (Real Scenario)

Suppose:

- An **EC2 instance needs to access an S3 bucket**.

Bad approach ❌

Store AWS credentials inside the EC2 instance.

Better approach ✅

Attach an **IAM Role** to the EC2 instance that has **S3 access permission**.

Then the EC2 instance can access S3 **securely without storing credentials**.

6. How EC2 Uses an IAM Role

Step-by-step process:

1. Create an **IAM Role**
2. Attach a **policy** (example: S3FullAccess)
3. Attach the role to the **EC2 instance**
4. EC2 automatically receives **temporary credentials**

Then you can run commands inside EC2 like:

```
aws s3 ls
```

Even though no credentials are stored in the instance.

7. Example IAM Role Policy

Example permission policy:

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": "s3:*",
   "Resource": "*"
  }
 ]
}
```

This role allows EC2 to access S3 services.

8. Access AWS from EC2 Using Role

Inside the EC2 instance:

```
aws ec2 describe-instances
```

or

```
aws s3 ls
```

AWS automatically uses **temporary credentials from the role**.

No need to configure:

```
aws configure
```

9. Why IAM Roles Are Better

Advantages:

- No need to store credentials
- Temporary credentials
- More secure
- Recommended by AWS

10. How to Delete AWS CLI Credentials

If credentials are stored locally and you want to remove them, you can delete them manually.

Step 1: Go to AWS CLI folder

Windows path:

```
C:\Users\YourUsername\.aws\
```

Step 2: Open credentials file

Example content:

```
[default]
aws_access_key_id = AKIA12345
aws_secret_access_key = abcxyz123
```

Step 3: Delete credentials

You can:

Option 1:

Delete the entire **credentials file**

OR

Option 2:

Remove the credential block

Example:

```
[default]
```

After deleting, AWS CLI will no longer have access to AWS services.

11. What Happens After Deleting Credentials

If you run a command like:

```
aws ec2 describe-instances
```

You will get an error:

```
Unable to locate credentials
```

This means AWS CLI cannot authenticate.

12. Best Security Practice

AWS recommends:

1. Avoid storing credentials on servers
2. Use **IAM Roles**
3. Use **temporary credentials**
4. Rotate access keys regularly

What is AWS CloudShell?

AWS CloudShell is a **browser-based terminal provided by AWS** that allows users to run **AWS CLI commands directly from the AWS Management Console**.

It is a **pre-configured shell environment** with AWS CLI already installed.

This means users can manage AWS services **without installing any software on their local computer**.

2. Key Features of AWS CloudShell

Main features of CloudShell:

- Browser-based terminal
- AWS CLI already installed
- No configuration required
- Secure access using IAM credentials
- 1 GB persistent storage

CloudShell supports common Linux tools like:

- bash
- python
- git
- npm
- pip

3. Why Use AWS CloudShell?

Normally to use AWS CLI you must:

1. Install AWS CLI
2. Configure credentials using `aws configure`

But with **AWS CloudShell**:

- AWS CLI is **already installed**
- AWS credentials are **automatically configured**

So users can start running AWS commands immediately.

4. How to Open AWS CloudShell

Steps to open CloudShell:

1. Login to **AWS Management Console**
2. Click the **CloudShell icon** in the top navigation bar
3. A terminal window will open in the browser

Now you can run AWS commands directly.

Example command:

```
aws ec2 describe-instances
```

5. Environment of AWS CloudShell

CloudShell runs on a **Linux-based environment**.

Features:

- Bash shell
- AWS CLI installed
- Python installed
- Git installed
- Docker CLI available

It behaves like a **Linux terminal inside your browser**.

6. Storage in CloudShell

AWS CloudShell provides **persistent storage**.

Details:

- 1 GB storage
- Files are stored in **home directory**

Example:

```
/home/cloudshell-user
```

Files saved here remain available even after closing CloudShell.

7. Example Commands in CloudShell

List S3 buckets:

```
aws s3 ls
```

List EC2 instances:

```
aws ec2 describe-instances
```

Create a directory:

```
mkdir project
```

List files:

```
ls
```

8. Advantages of AWS CloudShell

Advantages include:

- No need to install AWS CLI
- Works from any browser
- Automatically authenticated
- Secure environment
- Easy for quick testing

9. Limitations of AWS CloudShell

Some limitations include:

- Limited storage (1 GB)
- Limited compute resources
- Temporary session environment

10. When to Use AWS CloudShell

CloudShell is useful when:

- You want to run quick AWS CLI commands
- You do not have AWS CLI installed
- You need a secure environment
- You want to test scripts quickly

11. CloudShell vs AWS CLI (Local)

Feature	AWS CLI (Local)	AWS CloudShell
Installation	Required	Not required
Configuration	Required	Automatic
Environment	Local machine	Browser-based
Storage	Local storage	1 GB persistent storage